

Lecture 2 Optimization Techniques

Basics of Gradient-Based Optimization

Gradient Descent

- Iterative update:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

- Key role of **learning rate** η :
 - If too high $\rightarrow$ oscillation/divergence.
 - If too low $\rightarrow$ painfully slow convergence.

Stochastic Gradient Descent (SGD)

- Instead of full dataset gradient, use minibatch:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L_{B_t}(\theta_t)$$

where L_{B_t} is the loss over minibatch B_t .

- **Benefits:**
 - Scales to large datasets.
 - Stochasticity helps escape shallow minima.

Why is it called *stochastic*?

Because each step uses a **randomly sampled** data point or mini-batch. That randomness makes the gradient **stochastic** (noisy).

- With a **constant** η , SGD hovers around a minimum due to gradient noise.
- With a **decaying** η_t (e.g., $\eta_t = \eta_0 / (1 + \lambda t)$), SGD converges (for convex problems) to the optimum in expectation.

Advanced Gradient-Based Methods

Momentum

Instead of updating only based on the current gradient, **Momentum** accumulates a moving average of past gradients to keep a “velocity.”

Equations:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- θ_t : model parameters at step t
- $L(\theta_t)$: loss function
- $\nabla_{\theta} L(\theta_t)$: gradient at step t
- v_t : velocity term (exponentially weighted moving average of past gradients)
- $\beta \in [0, 1)$: momentum coefficient (usually 0.9)
- η : learning rate

Nesterov Accelerated Gradient (NAG)

A refinement of Momentum: looks **ahead** before computing gradient.

Equations:

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} L(\theta_t - \eta \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- $\eta\beta v_{t-1}$ = a "predicted step" based on **past momentum**.
- $\theta_t - \eta\beta v_{t-1}$ = **lookahead position**, i.e., "where the parameters **would be** if we just continued momentum without looking at the current gradient."

Adagrad

Adapts learning rates per parameter based on historical squared gradients.

Equations:

$$G_t = \sum_{k=1}^t \nabla_{\theta} L(\theta_k) \odot \nabla_{\theta} L(\theta_k)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot \nabla_{\theta} L(\theta_t)$$

- G_t : sum of past squared gradients (per parameter)
- $\odot$: elementwise multiplication/division
- ϵ : small constant to avoid division by zero

RMSProp

Fixes Adagrad's decaying LR by using an **exponential moving average** instead of cumulative sum.

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)(\nabla_{\theta} L(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \odot \nabla_{\theta} L(\theta_t)$$

Adam (Adaptive Moment Estimation)

Combines **Momentum + RMSProp**.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

- m_t : first moment estimate (like momentum)
- v_t : second moment estimate (like RMSProp)
- β_1, β_2 : decay rates (e.g., 0.9, 0.999)
- Bias correction ($\hat{m}_t, \hat{v}_t$) $\rightarrow$ removes initialization bias.

Second-Order & Advanced Methods

Unlike first-order (gradient only), **second-order methods** use curvature (Hessian matrix) for better updates.

Newton's Method

$$\theta_{t+1} = \theta_t - H_t^{-1} \nabla_{\theta} L(\theta_t)$$

- $H_t = \nabla_{\theta}^2 L(\theta_t)$: Hessian matrix (matrix of second derivatives).
- H_t^{-1} : inverse Hessian.

Quasi-Newton (BFGS, L-BFGS)

Approximate Hessian without computing it directly.

- **BFGS**: builds up inverse Hessian approximation iteratively.
- **L-BFGS**: limited-memory version, keeps only a few vectors, used for large-scale optimization.

Equation (L-BFGS update is iterative, simplified here):

$$H_{t+1} \approx H_t + \frac{y_t y_t^T}{y_t^T s_t} - \frac{H_t s_t s_t^T H_t}{s_t^T H_t s_t}$$

- $s_t = \theta_{t+1} - \theta_t$
- $y_t = \nabla L(\theta_{t+1}) - \nabla L(\theta_t)$

Broyden-Fletcher-Goldfarb-Shanno.

1. Define:

$$s_t = \theta_{t+1} - \theta_t$$

$$y_t = \nabla L(\theta_{t+1}) - \nabla L(\theta_t)$$

- s_t = step taken in parameter space
- y_t = change in gradient

2. Update inverse Hessian approximation H_{t+1} :

$$H_{t+1} = \left(I - \frac{s_t y_t^T}{y_t^T s_t} \right) H_t \left(I - \frac{y_t s_t^T}{y_t^T s_t} \right) + \frac{s_t s_t^T}{y_t^T s_t}$$

- H_t = current Hessian approximation
- I = identity matrix
- $s_t y_t^T / y_t^T s_t$ = rank-one correction
- $s_t s_t^T / y_t^T s_t$ = ensures **positive definiteness**

3. Parameter update:

$$\theta_{t+1} = \theta_t - H_{t+1} \nabla L(\theta_t)$$

How L-BFGS works:

1. Keep a queue of last m pairs (s_t, y_t) .
2. Use these to **implicitly approximate the Hessian** when computing the search direction.
3. Memory and computation scale as $O(nm)$ instead of $O(n^2)$.

Natural Gradient Descent (NGD)

Instead of using Euclidean distance in parameter space, uses **Fisher Information Matrix (FIM)** to measure distance in distribution space.

$$\theta_{t+1} = \theta_t - \eta F^{-1} \nabla_{\theta} L(\theta_t)$$

- F : Fisher Information Matrix (captures geometry of parameterized probability distribution).

The FIM measures **how much information a random variable contains about an unknown parameter θ** .

Suppose we have a parametric probability model $p(x; \theta)$ (e.g., output of a neural network with softmax).

The **FIM** is:

$$F(\theta) = \mathbb{E}_{x \sim p(x; \theta)} \left[\nabla_{\theta} \log p(x; \theta) \nabla_{\theta} \log p(x; \theta)^T \right]$$

Where:

- $\nabla_{\theta} \log p(x; \theta)$ = **score function** (gradient of log-likelihood w.r.t parameters)
- Outer product $\nabla_{\theta} \log p \nabla_{\theta} \log p^T$ gives a **matrix**
- Expectation $\mathbb{E}_{x \sim p(x; \theta)} [\cdot]$ = average over data drawn from the model

